

— NATURAL LANGUAGE PROCESSING

Text Preprocessing

— Chapter 2

Course: Natural Language Processing (NLP)

Audience: Third-Year Artificial Intelligence Students

Reference: Jurafsky & Martin — Speech and Language Processing, 3rd Ed.



| Chapter Learning Outcomes

After completing this chapter, students should be able to:

01 Explain the importance of text preprocessing in NLP

02 Apply text cleaning techniques to raw textual data

03 Understand tokenization and distinguish between approaches

04 Explain stopword removal and when to apply it

05 Describe stemming algorithms and how they work

06 Explain lemmatization and compare it with stemming

07 Implement all preprocessing techniques using Python and NLTK

| Why Do We Need Preprocessing?

Human language is rich, ambiguous, and variable. Computers see text literally.

HUMAN READS THESE AS IDENTICAL

Apple is a great company.

APPLE IS A GREAT COMPANY!!

apple is a great company

COMPUTER SEES THEM AS DIFFERENT

3 different character sequences → 3 different tokens → 3 different features

Text Preprocessing transforms raw text into a cleaner, more consistent representation before any NLP algorithm is applied.

WITHOUT PREPROCESSING, NLP MODELS SUFFER FROM:

- Increased vocabulary size
- Higher computational cost
- More noisy features
- Reduced model accuracy
- Poor generalization

The NLP Preprocessing Pipeline

Each stage produces output that feeds the next. Together they transform noisy text into structured data.



Raw Text



Cleaning



Tokenization



Stopword
Removal

Remove noise

URLs, emails, punctuation, numbers, special characters

Split into units

Words, sentences, or characters

Reduce vocabulary

Remove frequent, low-information words

I Running Example

We will use this sentence throughout the chapter to illustrate every preprocessing step.

The students are studying Artificial Intelligence at the University!!

Each preprocessing step will transform this sentence:

After Cleaning

the students are studying artificial intelligence at the university

After Stopword Removal

['students', 'studying', 'artificial', 'intelligence', 'university']

After Lemmatization

['student', 'study', 'artificial', 'intelligence', 'university']

Why a running example? It lets you clearly observe the cumulative effect of each operation rather than starting fresh with each section.

| The Vocabulary Problem

Spam detection example — should these be four different features?

BEFORE PREPROCESSING

- FREE
- Free
- free
- FREE!!

Vocabulary Size: 4

Preprocessing



AFTER PREPROCESSING

- free

Vocabulary Size: 1

SMALLER VOCABULARY MEANS:

- Less memory consumption
- Faster model training
- Better statistical estimation
- Reduced feature sparsity

| The Five Main Preprocessing Steps

Each technique solves a different problem. Together they transform noisy raw text into structured data.

1

Cleaning

Remove URLs, emails, numbers, punctuation, and standardize case

2

Tokenization

Divide text into meaningful units
— words, sentences, or characters

3

Stopword Removal

Eliminate high-frequency words with little semantic value

4

Stemming

Reduce words to approximate stems by removing common suffixes

5

Lemmatization

Map words to their canonical dictionary form using linguistic knowledge

Note: Not all steps are required for every NLP task. The choice of preprocessing steps depends on the specific application.

— SECTION 1 —

Text Cleaning

Lowercasing

Punctuation

URLs & Emails

Special Characters

Numbers

| What is Text Cleaning?

Definition

Text Cleaning removes unnecessary or noisy elements from raw text while preserving useful information for the NLP task.

RAW TEXT FROM THE WEB CONTAINS:

- Symbols & special characters
- Repeated punctuation
- Hyperlinks & HTML tags
- Inconsistent capitalization
- Numbers & codes
- Email addresses

OBJECTIVES OF TEXT CLEANING:

- | Reduce textual noise
- | Standardize writing style
- | Reduce vocabulary size
- | Improve computational efficiency
- | Improve feature quality
- | Prepare for downstream stages

Lowercasing

Convert all text to lowercase to treat identical words as the same token.

BEFORE — 4 DIFFERENT TOKENS

Apple

APPLE

apple

ApPIE

.lower()

AFTER — 1 TOKEN

apple

Advantages

- Reduces vocabulary size
- Improves consistency
- Simplifies feature extraction

Limitation — Context Matters

Named Entity vs. Common Word:

US → United States

us → pronoun

Brand vs. Object:

Apple → tech company

apple → the fruit

Rule: Always evaluate whether case carries semantic meaning for your specific NLP task before applying lowercasing.

Removing Punctuation

COMMON PUNCTUATION MARKS

. , ! ? ; : () " ' - /

BEFORE

I love NLP!!!

```
re.sub(r'^a-zA-Z\s', "", text)
```

AFTER

I love NLP

Semantic meaning preserved

Advantages

- Removes unnecessary symbols
- Simplifies tokenization
- Reduces feature sparsity

When to Keep Punctuation

In some applications, punctuation carries important meaning:

Really? — skepticism

Really! — excitement

Tasks: Sentiment Analysis, Question Answering, Chatbots

Removing Numbers & URLs

Numbers

Whether to remove depends on the application:

Remove When

Topic classification
Document clustering
Spam detection

Keep When

Financial documents
Medical reports
Scientific papers

```
re.sub(r'\d+', '', text)
```

URLs

Before:

Visit <https://openai.com>

```
re.sub(r'https?: //\S+', '', text)
```

After:

Visit

Email Addresses

Email: ahmed@gmail.com

```
re.sub(r'\S+@\S+', '', text)
```

Email:

Extra Spaces

"I love NLP"

```
re.sub(r'\s+', ' ', text).strip()
```

"I love NLP"

Key principle: The appropriate cleaning steps always depend on the NLP task requirements.

| Complete Cleaning Pipeline — Step by Step

Starting text: Hello!!! My Name Is Ahmed. Visit <https://www.example.com> Email me at ahmed@gmail.com I LOVE NLP!!!

Step 1 Lowercase

```
hello!!! my name is ahmed. visit https://www.example.com email me
at ahmed@gmail.com i love nlp!!!
```

Step 2 Remove URL

```
hello!!! my name is ahmed. visit email me at ahmed@gmail.com i
love nlp!!!
```

Step 3 Remove Email

```
hello!!! my name is ahmed. visit email me at i love nlp!!!
```

Step 4 Remove Numbers

```
hello!!! my name is ahmed. visit email me at i love nlp!!!
```

Step 5 Remove Punctuation & Extra Spaces

```
hello my name is ahmed visit email me at i love nlp
```

FINAL CLEAN OUTPUT

```
hello my name is ahmed
visit email me at i love
nlp
```

Notice: essential meaning is preserved while all noise is removed.

Important Reminder

No universal cleaning pipeline exists. Always select steps based on your specific NLP task requirements.

I Common Cleaning Mistakes

Critical Misconception: Cleaning does NOT always improve performance. The right strategy depends entirely on the application.



Removing Punctuation from Sentiment Analysis

Punctuation marks like ! and ? carry emotional meaning. Removing them reduces model performance.

Really? vs Really! — completely different intentions



Removing Numbers from Financial Documents

Numbers are the core information in financial reports, medical data, and scientific papers.

"Revenue increased 42%" — removing 42 destroys the meaning



Lowercasing Named Entities

Capitalization often encodes semantic meaning for named entity recognition and disambiguation.

Apple (company) vs apple (fruit) — case is the only differentiator

Golden Rule: Always ask — what information does this element carry for my specific task? Remove it only if the answer is "none".

— SECTION 2 —

Tokenization

Sentence Tokenization

Word Tokenization

Character Tokenization

Challenges

| What is Tokenization?

Definition

Tokenization is the process of splitting a sequence of text into smaller units called **tokens**.

COMPUTER SEES RAW TEXT AS ONE CHARACTER STRING:

Artificial Intelligence is transforming healthcare.

AFTER TOKENIZATION — A STRUCTURED LIST:

'Artificial' 'Intelligence' 'is' 'transforming'
'healthcare' '.'

A TOKEN MAY REPRESENT:

A word A sentence Punctuation A number
A character

USED IN ALMOST EVERY NLP APPLICATION:

- Information Retrieval
- Search Engines
- Machine Translation
- Sentiment Analysis
- Question Answering
- Chatbots & Language Models
- Text Classification

Three Types of Tokenization

Different strategies produce different granularities. Choice depends on the application.

S Sentence Tokenization

Divides a document into individual sentences.

```
sent_tokenize(text) →  
"NLP is exciting."  
"It combines linguistics  
and AI."  
"Many apps rely on NLP."
```

Used in: summarization, translation, dialogue systems

W Word Tokenization

Divides each sentence into individual words.

```
word_tokenize("NLP is  
amazing.") →  
'NLP' 'is' 'amazing'  
'.'
```

Most common type — the foundation of NLP features

C Character Tokenization

Divides text into individual characters.

```
"NLP" →  
'N' 'L' 'P'
```

Used in: OCR, spelling correction, character-level models

Comparing Tokenization Types

Input text: "I love NLP."

Sentence

1 token — entire input is one sentence

```
["I love NLP."]
```

Granularity: Document-level
Use: Segmentation, translation

Word

4 tokens — words and punctuation separated

```
'I'  
'love'  
'NLP'  
'.'
```

Granularity: Word-level
Use: Classification, sentiment

Character

11 tokens — every character is a unit

```
'I' ' ' 'l' 'o' 'v' 'e'  
' ' 'N' 'L' 'P' '.'
```

Granularity: Character-level
Use: OCR, spell correction

Key Insight: The same text produces completely different outputs depending on the tokenizer. There is no universally "correct" tokenization — it depends on the task.

| Tokenization Challenges

Natural language contains many special cases that make tokenization surprisingly difficult.

Contractions

How should contractions be split?

```
"I'm studying NLP."
```

Option A: ['I', "'m'] or Option B: ['I', 'am']

Abbreviations

Periods inside abbreviations are not sentence boundaries.

```
"Dr. Smith lives in the U.S.A."
```

Must recognize Dr. and U.S.A. as single tokens

Decimal Numbers

The period in a number is not a sentence boundary.

```
"The price is 19.95 dollars."
```

Should be: ['19.95'] not ['19', '95']

Hyphenated Words & Emails

Keep together or split?

```
"state-of-the-art" |
```

```
"support@example.com"
```

Context and application determine the answer

Solution approaches: Rule-based tokenizers use handcrafted rules. Statistical tokenizers learn token boundaries from annotated corpora. NLTK's `word_tokenize` handles most English edge cases automatically.

— SECTION 3 —

Stopword Removal

What are Stopwords?

Why Remove?

When to Keep?

I What are Stopwords?

Definition

A **Stopword** is a high-frequency word that carries little semantic information for a particular NLP task. Also called **function words** — grammatical rather than semantic role.

EXAMPLE — BEFORE VS AFTER REMOVAL:

Original:

The students are studying Artificial Intelligence in the university.

After removal:

['students', 'studying', 'artificial', 'intelligence', 'university']

COMMON ENGLISH STOPWORDS:

a an the is am are was were
be been to of for on in at
by from with and or but if
this that these those

Note: Words like "the", "are", "in" appear in almost every document regardless of topic — they cannot help distinguish one document from another.

| Why Remove Stopwords?

Consider two documents. Which words distinguish their topics?

Document A

The **students** are **studying Artificial Intelligence**.

Document B

The **doctors** are **studying medicine**.

Words shared by both — not useful for distinction:

the **are**

Informative words that distinguish topics:

students **Artificial** **Intelligence** **doctors**
medicine

THREE MAIN ADVANTAGES:

1. Smaller Vocabulary

Common words like "the", "is", "of" dominate frequency counts. Removing them reduces vocabulary significantly.

2. Faster Training

Fewer words means fewer features, leading to faster processing and shorter model training time.

3. Better Feature Extraction

Frequency-based features like TF-IDF become more meaningful when stopwords no longer dominate the counts.

| When to Remove vs. Keep Stopwords

Stopword removal is NOT always beneficial. The decision is entirely task-dependent.

Remove Stopwords

Tasks focused on identifying the main topic:

- Text Classification
- Spam Detection
- Topic Modeling
- Information Retrieval
- Keyword Extraction
- Document Clustering

Keep Stopwords

Tasks requiring complete grammatical sentences:

- Machine Translation
- Question Answering
- Text Summarization
- Grammar Checking
- Dialogue Systems
- Large Language Models

I The "NOT" Problem in Sentiment Analysis

ORIGINAL SENTENCE

I do **not** like this movie.

After removing stopwords "not":



AFTER STOPWORD REMOVAL

I like this movie.

Sentiment completely reversed!

Negative review → classified as positive → wrong prediction

Negation Words as Stopwords?

Negation words are often included in stopwords lists but carry critical semantic weight:

not no never neither nor
nothing nobody

Best Practice

For Sentiment Analysis, always remove negation words from your stopwords list, or do not apply stopwords removal at all.

APPLICATIONS WHERE STOPWORD REMOVAL IS RISKY:

- Sentiment Analysis
- Fake News Detection
- Fact Verification

— SECTION 4 —

Stemming

Suffix Stripping

Porter Stemmer

Pros & Cons

| What is Stemming?

Definition

Stemming is the process of reducing different grammatical forms of a word to a common base form called the **stem**. A stem is NOT necessarily a valid dictionary word.

EXAMPLE — PLAY AND ITS FORMS:

BEFORE		AFTER
playing		play
played	→	play
plays		play
player		player

WHY DO WE NEED STEMMING?

Without stemming, a search for "connect" may miss documents containing "connected", "connecting", or "connection". With stemming, all are reduced to approximately the same stem, improving recall.

MORPHOLOGICAL VARIATIONS

Base Word

learn
study
organize
develop

Variations

learning, learned
studies, studied
organized,
organizing
developing,
developed

Core Principle

Stemming uses rule-based suffix stripping — no dictionary lookup required. This makes it fast but sometimes imprecise.

| How Does Stemming Work?

Most stemming algorithms use rule-based suffix stripping — no dictionary lookup required.

COMMON SUFFIXES REMOVED:

-ing -ed -es -s -ly -ment -tion -er -ness

Step-by-Step Examples

playing — remove -ing → play

walked — remove -ed → walk

studies — remove -ies → studi not a real word!

cars — remove -s → car

Advantages

- Fast — no dictionary lookup
- Memory efficient
- Reduces vocabulary size
- Improves search recall

Limitations

- May produce non-words
- Over-stemming possible
- Under-stemming possible

Key Rule: A stem does NOT need to be a real word. It only needs to be consistent — group related forms together.

| Porter Stemmer

Developed by Martin Porter in 1980. The most widely used stemming algorithm in NLP.

Why Porter Stemmer?

- Simple to understand and implement
- Fast execution speed
- Memory efficient
- No dictionary required

Examples — Porter Stemmer Output

INPUT	STEM
caresses	caress
ponies	poni
studies	studi
relational	relat

Notice: "studi" and "poni" are not real English words!

This is perfectly acceptable — stems aim for consistency, not linguistic correctness. Their effectiveness is measured by how well they group related word forms.

Comparing Algorithms

Porter

Fast, moderate aggressiveness. Best balance for most applications.

Lancaster

Very fast but very aggressive. Often removes too many characters:
"maximum" → "maxim"

Snowball

Improved Porter. Supports multiple languages and provides better consistency.

| Stemming in the Full Pipeline

Watch the running example transform through each stage:

ORIGINAL

The students are studying Artificial Intelligence.

↓ Tokenization

TOKENIZED

['the', 'students', 'are', 'studying', 'artificial', 'intelligence']

↓ Stopword Removal

NO
STOPWORDS

['students', 'studying', 'artificial', 'intelligence']

↓ Porter Stemmer

STEMMED

['student', 'studi', 'artifici', 'intellig']

Observe: "artificial" → "artifici" and "intelligence" → "intellig" — neither is a valid English word. But they are consistent stems useful for retrieval and classification.

— SECTION 5 —

Lemmatization

Dictionary Forms

WordNet Lemmatizer

vs Stemming

| What is Lemmatization?

Definition

Lemmatization is the process of finding the **lemma** — the base dictionary form of a word. Unlike stemming, it uses linguistic knowledge to return a real, valid word.

EXAMPLE — VERB CONJUGATIONS:

Verb forms → lemma "run"

running → **run**

ran → **run**

runs → **run**

Noun forms → lemma

better → **good**

mice → **mouse**

worst → **bad**

children → **child**

Key Distinction from Stemming

Lemmatization always returns a real dictionary word. Stemming may not. This requires looking up words in a lexical database such as WordNet.

HOW IT WORKS:

- 1 Identify the Part of Speech (POS) of the word
- 2 Look up the word in a morphological lexicon (WordNet)
- 3 Return the canonical base form

POS Context Matters

meeting → **meet** (verb)

meeting → **meeting** (noun)

Stemming vs. Lemmatization

WORD	STEM (PORTER)	LEMMA (WORDNET)
studying	studi	study
running	run	run
better	better	good
mice	mice	mouse
caring	care	care
connections	connect	connection
generously	generous	generously

Stemming

- Fast, no dictionary needed
- May return non-real words
- Good for IR and classification

Lemmatization

- Slower, requires dictionary lookup
- Always returns valid words
- Better for high-precision tasks

| When to Use Stemming vs. Lemmatization

Choose Stemming

When speed and efficiency are priorities:

- **Large-scale text classification**
Processing millions of documents
- **Information retrieval**
Search engines with real-time requirements
- **Spam detection**
High-throughput email filtering
- **Prototype development**
Quick experiments without dictionary
- **Resource-constrained environments**
Mobile or embedded applications

Choose Lemmatization

When linguistic accuracy matters:

- **Chatbots and virtual assistants**
Need human-readable, valid words
- **Text summarization**
Must produce grammatically correct output
- **Question answering**
Precision is critical for correctness
- **Named entity recognition**
Word forms must be properly understood
- **Explainable AI systems**
Results must be interpretable to humans

Practical rule: If you need speed and can tolerate non-words → Stemming. If you need linguistic accuracy and valid vocabulary → Lemmatization. Modern large language models often skip both and use subword tokenization instead.

Python Lab

Hands-On with NLTK

Setup & Imports

Text Cleaning

Tokenization

Stopword Removal

Stemming

Lemmatization + POS

Setup & Imports

CODE

```
import nltk
import re
import string

nltk.download('punkt')
nltk.download('stopwords')
nltk.download('wordnet')
nltk.download('averaged_perceptron_tagger')

from nltk.tokenize import word_tokenize, sent_tokenize
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer
from nltk.stem import WordNetLemmatizer
from nltk.corpus import wordnet
```

What Each Import Does

nltk

Natural Language Toolkit — Python's main NLP library

re

Regular expressions for text pattern matching and cleaning

PorterStemmer

The most commonly used stemmer for English

WordNetLemmatizer

Lemmatizer backed by the WordNet lexical database

Text Cleaning

CODE

```
def clean_text(text):  
    # 1. Lowercase  
    text = text.lower()  
  
    # 2. Remove URLs  
    text = re.sub(r'https?: //\S+', '', text)  
  
    # 3. Remove emails  
    text = re.sub(r'\S+@\S+', '', text)  
  
    # 4. Remove numbers  
    text = re.sub(r'\d+', '', text)  
  
    # 5. Remove punctuation  
    text = re.sub(r'[^\\w\\s]', '', text)  
  
    # 6. Remove extra whitespace  
    text = re.sub(r'\s+', ' ', text).strip()
```

Expected Output

hello visit email
results

Function Breakdown

`.lower()` Lowercase all
text

`re.sub` Pattern
substitution

`r'\S+@\S+'` Email regex
pattern

`.strip()` Remove
leading/trailing
space

Tip: Order matters!
Always remove URLs
and emails before
removing punctuation
to avoid partial

Tokenization

CODE

```
# Sample text
text = "NLP is fascinating. It enables machines to understand language."

# Sentence tokenization
sentences = sent_tokenize(text)
print("Sentences:", sentences)

# Word tokenization
words = word_tokenize(text)
print("Words:", words)

# Character tokenization
chars = list(text)
print("First 10 chars:", chars[:10])
```

Expected Output

Sentences:

```
['NLP is
fascinating.', 'It
enables machines to
understand
language.']
```

Words:

```
['NLP', 'is',
'fascinating', '.',
'It', 'enables',
...]
```

First 10 chars:

```
['N','L','P','
','i','s','
','f','a','s']
```

Note

NLTK's word_tokenize also separates punctuation marks as individual tokens. This

Stopword Removal & Stemming

CODE

```
# Get English stopwords
stop_words = set(stopwords.words('english'))

text = "The students are studying Artificial Intelligence"
words = word_tokenize(text.lower())

# Remove stopwords
filtered = [w for w in words
            if w not in stop_words
            and w.isalpha()]

print("After stopwords removal:", filtered)

# Porter Stemmer
ps = PorterStemmer()
stemmed = [ps.stem(w) for w in filtered]
print("After stemming:", stemmed)
```

Expected Output

```
After stopwords removal:
['students', 'studying',
 'artificial', 'intelligence']

After stemming:
['student', 'studi',
 'artifici', 'intellig']
```

Key Code Points

set() — converts list to set for O(1) lookup speed

.isalpha() — also removes punctuation tokens

ps.stem(w) — applies Porter stemming rules

Remember: "studi" and "artifici" are not real words but are consistent stems — valid for IR and classification tasks.

Lemmatization with POS Tags

CODE

```
lemmatizer = WordNetLemmatizer()

# Without POS tag (defaults to noun)
print(lemmatizer.lemmatize('running'))
# Output: 'running' (wrong!)

# With POS tag (verb)
print(lemmatizer.lemmatize('running', pos='v'))
# Output: 'run' (correct!)

# More examples
print(lemmatizer.lemmatize('better', pos='a'))
# Output: 'good'
print(lemmatizer.lemmatize('mice', pos='n'))
# Output: 'mouse'
print(lemmatizer.lemmatize('worst', pos='a'))
# Output: 'bad'
```

POS Tags in NLTK

'n' Noun

'v' Verb

'a' Adjective

'r' Adverb

Critical: POS Tag is Required

Without a POS tag, the lemmatizer defaults to treating words as nouns. This produces incorrect lemmas for verbs and adjectives. Always provide the correct POS tag for best results.

Final Comparison: All Five Techniques

Key Takeaway: These five techniques form a standard text preprocessing pipeline. The pipeline is modular — each step is optional and should be chosen based on the requirements of the downstream NLP task.

Lab Exercise

Task

Apply the complete preprocessing pipeline to the following text and report the output after each stage.

INPUT TEXT

"Machine Learning is TRANSFORMING our world!!!
Visit <https://ml.example.com> for more info.
Contact ml@support.com. The 3 main techniques
are classification, regression, and
clustering."

STEPS TO COMPLETE

- 1 Clean: Lowercase, remove URL, email, numbers, punctuation, extra spaces
- 2 Tokenize: Word tokenization using `word_tokenize()`
- 3 Remove Stopwords: Use NLTK English stopwords list
- 4 Apply Stemming: Use PorterStemmer

Expected Outcomes to Analyze

After Cleaning

All lowercase, no URLs, emails, numbers, or punctuation

Key Words After Stopwords

machine, learning,
transforming, world, main,
techniques, classification,
regression, clustering

Discuss

How does "transforming" differ between stemming output and lemmatization output?

Chapter Summary: Text Preprocessing Pipeline

1 **1. Text Cleaning** Remove noise: lowercase, strip URLs, emails, numbers, punctuation, extra spaces.

2 **2. Tokenization** Split text into tokens: sentence, word, or character level depending on the task.

3 **3. Stopword Removal** Remove high-frequency words carrying no semantic value for the target task.

4 **4. Stemming** Reduce word forms to an approximate stem using suffix-stripping rules.

5 **5. Lemmatization** Reduce word forms to a valid dictionary lemma using linguistic knowledge.

Core Principle: Every preprocessing decision must be justified by the requirements of your specific NLP task. There is no single correct pipeline.

| Key Takeaways



No Universal Pipeline

Text preprocessing is task-dependent. The same step that helps one application may harm another. Always analyze your specific use case.



Preprocessing Reduces Noise

Cleaning, tokenization, and stopwords removal together significantly reduce vocabulary size and feature sparsity, improving model efficiency.



Stemming = Speed

Stemming is fast and requires no dictionary. It may produce non-words, but produces consistent groupings useful for IR and classification.



Lemmatization = Precision

Lemmatization returns valid words using linguistic knowledge. It is slower but more accurate — essential when output must be human-readable.



Stopwords: Context-Dependent

Stopwords are not always noise. Negation words like "not" are crucial for sentiment analysis. Always audit your stopwords list for the task.



Tokenization Granularity Matters

Sentence, word, and character tokenization produce fundamentally different representations. The right granularity depends on the application.

I Quiz Questions

- 1 What is the main difference between stemming and lemmatization? Give an example using the word "running". [3 marks]
- 2 Why would removing stopwords HARM performance in a machine translation task? Explain with a specific example. [3 marks]
- 3 Given the sentence: "The Best ML models are training on GPU clusters." Apply the full preprocessing pipeline: clean, tokenize, remove stopwords, and stem. Show the output after each step. [5 marks]
- 4 A company wants to build a customer review sentiment classifier. Which preprocessing steps would you apply or avoid, and why? [4 marks]
- 5 What happens if you apply lowercasing before named entity recognition (NER)? Explain the potential problem with a concrete example. [3 marks]

Total: **18 marks** | Time: **20 minutes**

Open notes, closed internet

— END OF CHAPTER 2 —

Thank You

Questions & Discussion

5 Techniques

Cleaning, Tokenization,
Stopwords, Stemming,
Lemmatization

1 Principle

Every decision is task-
dependent.
No single correct
pipeline exists.

Next Chapter

Feature Extraction:
Bag of Words & TF-IDF

Introduction to Natural Language Processing

Third Year — Bachelor of Artificial Intelligence